



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Caracterización de cargas de trabajo en el microcontrolador SIWA (RISC-V RV32I en FPGA)

Bitácora 2

Proyecto Final de Graduación

Desarrollado en:	DCI-LAB, Laboratorio de Diseño de Circuitos Integrados
Institución:	Instituto Tecnológico de Costa Rica
Estudiante(s):	Iván Loaiza Arce
Carné:	2018086515
Profesor asesor actual:	PhD. Luis C. Rosales Alpizar
Fecha:	21 de marzo de 2026

Introducción

El presente documento corresponde a la bitácora de actividades del proyecto relacionado con el microcontrolador SIWA basado en arquitectura RISC-V. En esta bitácora se registran reuniones, avances técnicos y decisiones relevantes tomadas durante el proceso de desarrollo, implementación y mejoramiento del sistema y de su entorno de compilación.

Registro de actividades

Día 1 – Viernes 13 de marzo de 2026

El día viernes 13 de marzo el equipo se reunió en el laboratorio DCI-Lab con el objetivo de continuar con las actividades de síntesis e implementación del microcontrolador SIWA en FPGA. Durante esta jornada, se estableció comunicación con el profesor tutor, así como con los profesores Renato Rimolo Donadio y Roberto Molina, debido a que los intentos previos de síntesis e implementación no habían sido exitosos.

Como resultado, se acordó realizar una reunión adicional el mismo día en horas de la noche con el profesor Roberto Molina, con el fin de analizar en detalle la arquitectura del sistema y comprender el flujo correcto de implementación.

Durante esta reunión, se obtuvo acceso al repositorio oficial y a la documentación técnica de referencia del SIWA, lo cual permitió aclarar aspectos clave de su arquitectura y funcionamiento.

Arquitectura del microcontrolador

Se analizó la estructura de interfaces del microcontrolador, la cual incluye tres módulos principales: SPI, UART y GPIO, como se muestra en la Figura 1.

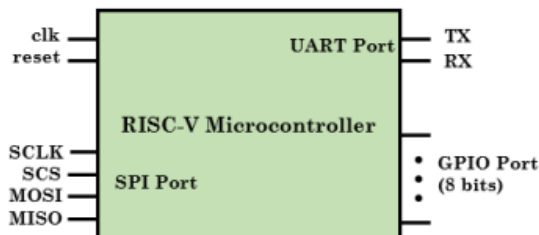


Figura 1: Interfaces principales del microcontrolador RISC-V.

El puerto SPI es utilizado para la comunicación con la memoria flash externa, donde se almacena el programa de arranque, mientras que el UART permite la comunicación con el PC y los GPIO se emplean para interacción con periféricos :contentReference[oaicite:0]index=0.

Plataforma de validación en FPGA

Se comprendió el enfoque de validación basado en FPGA, donde el sistema completo es emulado para pruebas funcionales, tal como se muestra en la Figura 2.

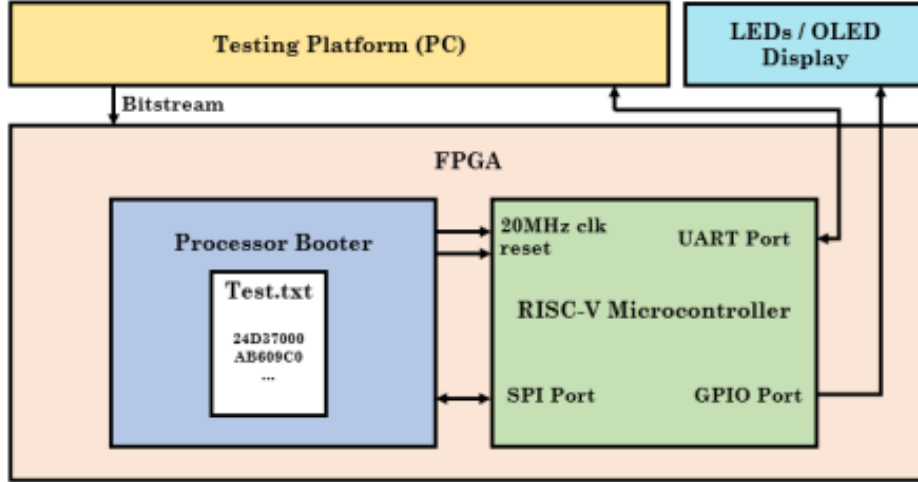


Figura 2: Plataforma de testing basada en FPGA.

En este esquema, el PC actúa como plataforma de control y monitoreo, mientras que la FPGA ejecuta tanto el núcleo RISC-V como los módulos de soporte, incluyendo el bootloader y los periféricos.

Arquitectura del bootloader

Un elemento clave identificado fue el módulo *bootloader*, cuya arquitectura se muestra en la Figura 3.

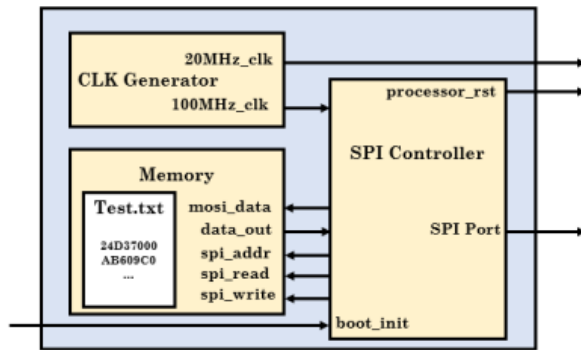


Figura 3: Arquitectura del módulo bootloader.

Este módulo es responsable de emular la memoria flash y gestionar el proceso de arranque del sistema, incluyendo la generación de reloj, control de reset y transferencia de instrucciones

al procesador mediante SPI.

Comunicación SPI y proceso de boot

Se analizó el comportamiento de la comunicación SPI durante el proceso de arranque, donde el microcontrolador envía comandos de lectura y recibe las instrucciones desde memoria, como se observa en la Figura 4.

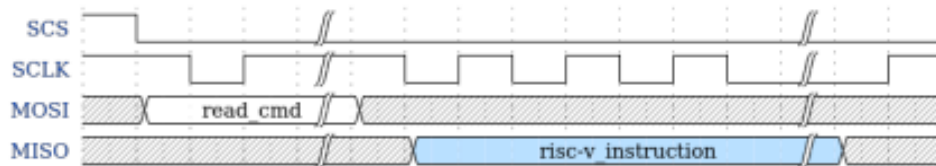


Figura 4: Diagrama de temporización de la comunicación SPI durante el boot.

En este proceso, el controlador SPI actúa como esclavo, interpretando los comandos recibidos por MOSI y respondiendo por MISO con los datos correspondientes, lo cual permite ejecutar el programa almacenado en la memoria externa :contentReference[oaicite:1]index=1.

A partir de este análisis, se identificó que los problemas previos de implementación estaban relacionados con una incorrecta integración del flujo de arranque y la interacción entre el controlador SPI y el núcleo del procesador.

Finalmente, se logró una alineación completa del equipo respecto a la arquitectura del sistema y su flujo de ejecución, permitiendo retomar las actividades de síntesis con una base técnica más sólida.

Adicionalmente, ese mismo día se actualizó el repositorio del proyecto, incorporando nueva documentación de referencia completamente dedicada al SIWA implementado, mejorando la trazabilidad y organización del desarrollo.

Día 2 – Martes 24 de marzo de 2026

El día martes 24 de marzo el equipo se reunió para revisar los avances relacionados con la síntesis del sistema SIWA.

Durante esta sesión se evidenciaron mejoras en el proceso de síntesis, logrando avances respecto a intentos anteriores. Sin embargo, aún no se había alcanzado una implementación funcional del sistema en la FPGA.

Se identificaron aspectos pendientes por resolver, principalmente relacionados con la correcta integración de los módulos y la carga del programa en memoria.

Día 3 – Jueves 26 de marzo de 2026

El día jueves 26 de marzo el equipo se reunió para evaluar los avances en la implementación del sistema SIWA en FPGA. Durante esta sesión se logró ejecutar el sistema siguiendo la estructura propuesta por el profesor Roberto Molina, lo cual permitió validar el flujo completo de integración del diseño en Vivado.

Se revisó la jerarquía del diseño dentro del entorno de desarrollo, identificando los módulos principales del sistema, tal como se muestra en la Figura 5.

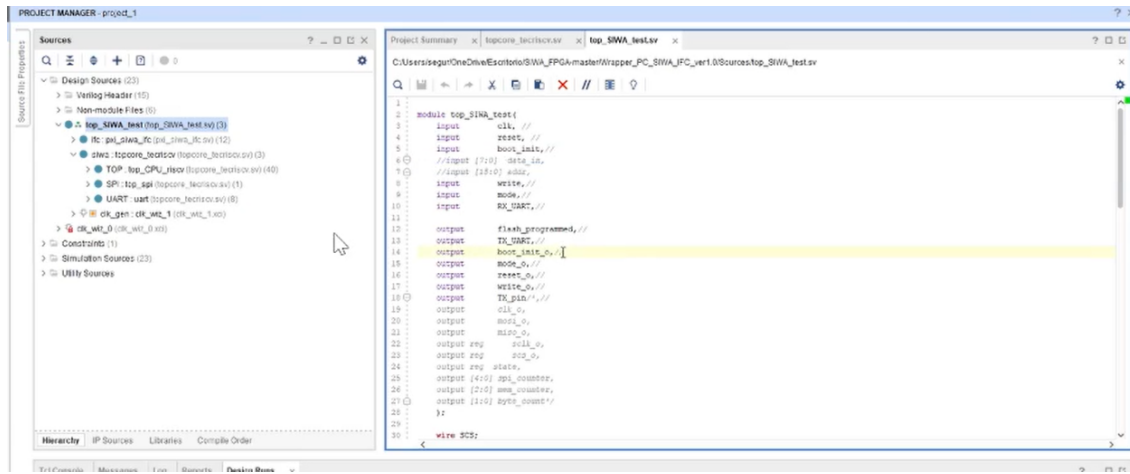


Figura 5: Jerarquía del diseño SIWA en Vivado.

En esta estructura se observa el módulo `top_SIWA_test`, el cual integra los bloques principales del sistema, incluyendo el núcleo del procesador RISC-V, los módulos de comunicación (SPI y UART) y los bloques auxiliares de control. Esta organización es clave para la correcta síntesis e implementación del sistema en FPGA.

Asignación de pines y constraints

Asimismo, se analizó el archivo de restricciones (.xdc), el cual define la asignación de pines físicos de la FPGA, como se muestra en la Figura 6.

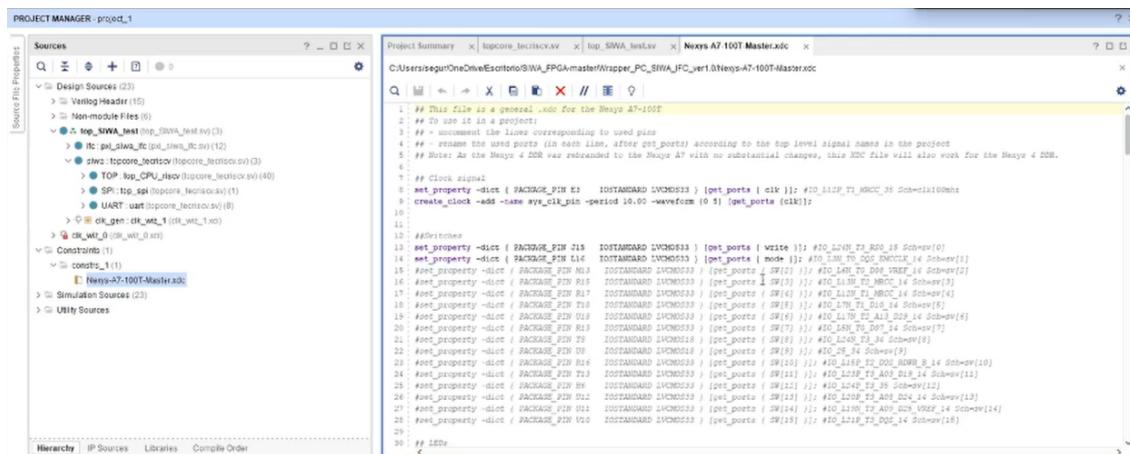


Figura 6: Archivo de restricciones para la FPGA Nexys A7.

Este archivo permite mapear señales como reloj, SPI, UART y GPIO hacia los pines físicos del dispositivo, siendo un elemento crítico para la correcta interacción entre el hardware y el

diseño implementado.

Implementación de la memoria SPI

Se revisó el módulo de memoria asociado al controlador SPI, el cual emula el comportamiento de la memoria flash, como se observa en la Figura 7.

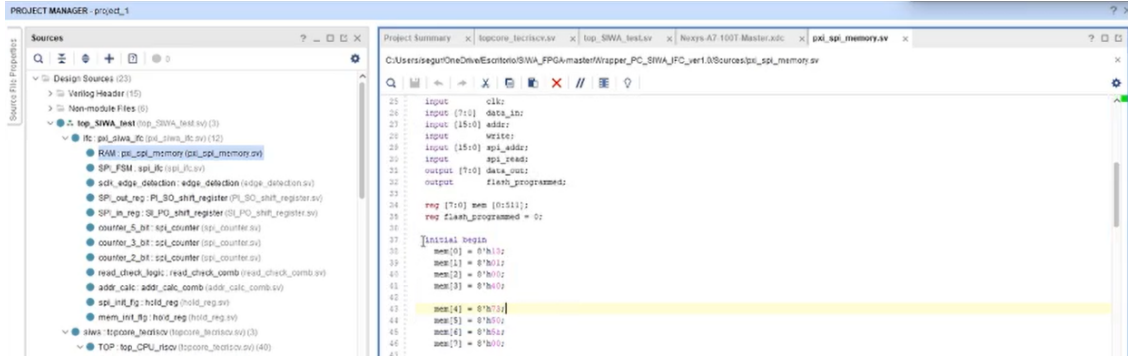


Figura 7: Implementación de memoria SPI en Verilog.

En este módulo se definen arreglos de memoria y lógica de lectura/escritura, permitiendo almacenar las instrucciones del programa que serán enviadas al procesador durante el proceso de arranque. Este bloque es fundamental para el funcionamiento del bootloader y la ejecución del sistema.

Análisis técnico

A partir de esta revisión, se logró comprender de manera integral cómo se encuentran conectados los diferentes módulos del sistema dentro de la FPGA, así como la importancia de:

- La correcta jerarquía del diseño.
- La asignación adecuada de pines mediante constraints.
- La implementación funcional de la memoria SPI.

Esto permitió consolidar la implementación del SIWA en FPGA y sentar las bases para continuar con el desarrollo de estrategias de evaluación del sistema.

Posteriormente, se inició el trabajo en nuevas líneas orientadas a la caracterización del sistema, incluyendo la definición de benchmarks, el desarrollo de testbench para medición de métricas y el análisis de métodos alternativos para la carga de programas en la FPGA, tales como el uso de memoria distribuida o memoria externa.

Día 4 – Viernes 27 de marzo de 2026

El día viernes 27 de marzo se llevó a cabo una reunión con el profesor tutor para presentar los avances obtenidos durante la semana.

Durante esta reunión se revisó el estado actual del proyecto y se contrastó con los objetivos definidos en la primera reunión. Se elaboró un checklist de seguimiento que permitió validar el cumplimiento de los puntos previamente establecidos.

Asimismo, se discutieron los siguientes pasos del proyecto, enfocados en la consolidación de la implementación del sistema, la mejora del flujo de trabajo y la formalización del proceso de medición de métricas mediante benchmarks.

Adicionalmente, se presentó la actualización del repositorio del proyecto, en el cual se incorporó nueva documentación de referencia completamente dedicada al SIWA implementado. Esta documentación permite una mejor comprensión del sistema en su estado actual y facilita la continuidad del desarrollo y la colaboración dentro del equipo.